

Algoritmos y Estructuras de Datos.

- Clase 2 - Excepciones y Funciones -

El commando Raise

Invocamos el comando `raise` de la siguiente forma:

```
raise NameError (string)
```

La ejecución del código se **detendra** de inmediato en la línea donde se ha llamado al comando `raise` y se escribirá el siguiente error `NameError (string)`.

Este comando nos sirve para "lanzar" errores, y avisar al sistema/usuario del evento.

In []:

```
# Ejemplo

n=int(input("Numerador : "))
d=int(input("Denominador : "))
if d==0:
    raise ValueError("El denominador debe ser distinto de '0'")
print(n*1.0/d)
```

También podemos hacer uso del comando `assert`, es muy similar al comando `raise`, de hecho podemos escribirlos casi de manera equivalente:

```
if someboolean :
```

```
    raise AssertionError ( string )
```

es equivalente a:

```
if :
```

```
    assert someboolean , string
```

Para que sirve cada uno y cuándo debemos usarlos?

- El comando `$\color{green}{\textsf{ try }}` nos permite manejar de errores, ya sea por propósitos de debugging del código, o dentro de funciones complejas propensas a generar errores. El uso de `$\color{green}{\textsf{ try }}` para el manejo de errores y nos permitiera ejecutar código específico para cada tipo de error. En caso de la terminación de un programa de manera inesperada, una buena práctica es intentar guardar el estado actual de la ejecución del programa, evitar más errores (e.g. cerrar archivos abiertos, desalocar memoria, etc.) y proveer una buena descripción del error que ha causado la terminación.
- El comando `$\color{green}{\textsf{ assert }}` es específico para un rápido debugging de nuestro código lanzando una excepción **AssertionError**. Debe ser usado cuando queremos encontrar un problema en nuestro código. El uso del comando `$\color{green}{\textsf{ assert }}` es generalmente considerado un "parche" y no una solución permanente. En comparación el comando `$\color{green}{\textsf{ try }}` podemos invocar nuestras excepciones y manejar los errores.
- El comando `$\color{green}{\textsf{ raise }}` es el cual nos permite lanzar (o invocar) excepciones para informar que se ha producido un error, y el "tipo" de error. Se debe destacar que las excepciones pueden ser definidas por el programador y al igual que en el caso del comando `$\color{green}{\textsf{ try }}`, las excepciones pueden contener código que permita manejar el error.

Funciones

Modificación de los argumentos

Dependiendo de si el argumento que le pasamos a la función es *mutable* o *no*, es posible modificar su contenido dentro de la función. A continuación veremos algunos ejemplos.

In []:

```
# Ejemplo definiendo la función 'incrementation'
def incrementation(x,k):
    x+=k
    return x
```

In []:

```
y=3
ret = incrementation(k=10,x=y) # Crea una variable 'x' con el valor de 'y', adic
iona 10; x+=10
                                     # No-mutable: pasaje de parametro "por copia".
ret
```

In []:

```
# Ejemplo
L=[1,'algo',3]
incrementation(L,[2]) # Crea una variable 'x' con el valor de 'L', luego agrega
                        # el 2; x+=2]
                        # Mutables: pasaje de parametro "por referencia"

L
```

In []:

```
# Ejercicio: Escribir una función que cambie todos los valores de un diccionario
por un 'string' nuevo.

def change(dico,c):
    for key in dico.keys():
        dico[key]=c
dico={'k1' : 'algo1', 'k2' : 'algo2'}

change(dico,'otrovalor')
```

In []:

```
dico
```

Cambiar un argumento mutable en la definición de la función puede ser propenso a "errores semanticos" (ya que estamos cambiando los valores internos de la variable/argumento).

Funciones Recursivas

Python nos permite una declaración sencilla de **funciones recursivas**, solamente debemos llamar a la función, dentro del cuerpo de la (misma) función. Debe destacarse, que siempre debemos tratar el caso base, o inicial de manera especial, ya que sino, la función podría no terminar. Por ejemplo: podemos manejar el caso base utilizando un `\normalsize \color{green}{\textsf{ if }}`, como veremos a continuación.

In []:

```
# Secuencia de Fibonacci.
def Fibo(n):
    "Definición: F(n)=F(n-1)+F(n-2), F(1)=F(0)=1"
    if n<0:
        raise ValueError("Fibonacci terms begin at 0") # sin este error, Fibo(-
1) entrara en un ciclo eterno.
    if n==0:
        return 1 # Primer caso base
    elif n==1:
        return 1 # Segundo caso base
    else:
        return Fibo(n-1)+Fibo(n-2) # Esta es la llamada recursiva
```

In []:

```
# Calculo recursivo del factorial de un número n.
def factorial(n):
    if type(n) is not int:
        raise ValueError("N debe ser un numero entero")
    elif n < 0 :
        raise ValueError("N debe ser mayor a 0")
    elif n<=1:
        return 1
    else:
        return factorial(n-1)* n
```

In []:

```
factorial(1.4)
```

In []:

```
[Fibo(n) for n in range(0,20)] # Para n>30 la performace es muy lenta.
```

El uso de funciones recursivas no debe ser abusado. Este tipo de funciones nos permite la facilidad de escritura de una función, aunque generalmente requieren el uso de mucha memoria. (Luego veremos cómo mejorar esto).

In []:

```
# Ejemplo: Fibonacci Iterativo
def Fiboiter(n):
    if n<0:
        raise ValueError("La secuencia de Fibonacci comienza en '0'") # si no co
mprabos que el parámetro es un número # entero,
por ejemplo, llamar a Fibo(-1) # causara
que la función nunca termine.
    elif n==0:
        return 1 # Primer caso base
    elif n==1:
        return 1 # Segundo caso base
    else:
        x=1 # elemento 0
        y=1 # elemento 1ro
        for i in range(1,n):
            # elemento 'x' i-1-esimo, e 'y' sera el i-esimo elemento
            x,y=y,x+y
            # elemento 'x' i-esimo, e 'y' sera el i+1-esimo elemento
        return y
```

In []:

```
N=10
print(Fibo(N), "=", Fiboiter(N))
```

In []:

```
Fiboiter(30)
```

Funciones y el "alcance" (scope) de las variables

El *alcance* de una variable $\texttt{v}$ es el conjunto de líneas de código, en donde la variable es comprendida. Es decir, donde el nombre de variable $\texttt{v}$ esta asignado a un valor. A priori, es simple: "si la variable $\texttt{v}$ esta definida en la línea n el alcance de la variable será cualquier línea $m > n$ ".

Esto se complica cuando introducimos funciones, estas pueden (o no) cambiar el alcance de una variable. Esta situación se complica aún más cuando tenemos *funciones anidadas*, es decir, funciones dentro de otras funciones. A continuación veremos algunos ejemplos:

In []:

```
# Investigar en los siguientes ejemplos: - qué será impreso por pantalla?
#                                         - cuál es el valor de x luego de cada l
#                                         - habrá algún mensaje de error?
#                                         - habrá algún mensaje de error?

x=2

def ExVar1():
    print(x)

def ExVar2():
    x=5
    print(x)

def ExVar3():
    x=5

    def ExVar11():
        global x
        print(x)

    ExVar11()
    print(x)

def ExVar4():
    print(x)
    print('x=1')
```

In []:

```
ExVar3()
```

La razón principal por la cual los variables tienen un alcance predefinido es para evitar **efectos secundarios**. Es decir, cambiar los valores de variables que pasamos por argumento y afectar el resto del código. Por ello, por defecto, el alcance de las variables es **dentro del cuerpo de la función**. Notar que aquellas variables **invocadas pero que no están en el cuerpo de la función no existiran**.

De esta manera podemos "proteger" las variables fuera de la función.

Es posible cambiar este comportamiento por defecto. Debemos usar el comando `\normalize \color{green}{\textsf{ global }}` o `\normalize \color{green}{\textsf{ nonlocal }}`. La diferencia entre ambos es sutil. Sea una variable `\texttt{v}`, invocando `\normalize \color{green}{\textsf{ global }} \texttt{v}` en el cuerpo de una función, `\texttt{v}` será usada como una variable global, perteneciente al código con mayor alcance. El comando `\normalize \color{green}{\textsf{ nonlocal }} \texttt{v}`, la variable `\texttt{v}` será aquella cual alcance es de **un nivel superior**, es decir, en funciones anidadas, aquella que tenga mayor alcance.

Debemos notar que al utilizar `\normalize \color{green}{\textsf{ nonlocal }}` en funciones recursivas, generara un error del tipo **SyntaxError**.

In []:

```
# Secuencia de Fibonacci contando la cantidad de llamadas recursivas
t=0
def Fibo(n):
    """n debe ser un número entero
    "F(n)=F(n-1)+F(n-2), F(1)=F(0)=1"""
    global t # la variable t será considerada como global
    t+=1     # hacemos algo con t
    if n<0:
        raise ValueError("Fibonacci terms begin at 0")
    elif n==0:
        return 1
    elif n==1:
        return 1
    else:
        return Fibo(n-1)+Fibo(n-2)
# Es posible estimar la cantidad de llamadas a la función?
```

In []:

```
help(Fibo)
```

In []:

```
t=0
2*Fibo(20)-1
```

In []:

```
t
```

In []:

```
# Diferencia entre global y no-local
def examplenothing():
    x='changedinexample'
    def insidenothing(): # inside... es una función definida dentro del alcance
de example...
        x='changedinside'
    insidenothing()
    print("la x en ejemplo... es ",x)

def exampleglobal():
    x='changedinexample'
    def insideglobal():
        global x          # x es una variable global en el alcance de insideglobal.
        x='changedinside'
    insideglobal()
    print("la x en ejemplo... es ",x)

def examplenonlocal():
    x='changedinexample'
    def insidenonlocal():
        nonlocal x        # x es una variable local a examplenonlocal
        x='changedinside'
    insidenonlocal()
    print("la x en ejemplo... es ",x)
```

In []:

```
x='notchanged'
examplenothing() # x no es cambiada en la primer función
print("global x es ",x) # x no es cambiada globalmente
print(5*'-')
x='notchanged'
exampleglobal() # x no es cambiada en la primer función
print("global x es ",x) # x cambiada globalmente
print(5*'-')
x='notchanged'
examplenonlocal() # x es cambiada en la primer función
print("global x es ",x) # x no es cambiada de forma global
print(5*'-')
```

In []:

```
# Diferencia entre global y no-local
def examplenothing():
    global x          #OK
    x='changedinexample'
    def insidenothing(): # inside... es una función definida dentro del alcance de example...
        x='changedinside' # x no es global dentro del alcance de insidenothing.
    insidenothing()
    print("la x en ejemplo... es ",x)

def exampleglobal():
    global x          # OK
    x='changedinexample'
    def insideglobal():
        global x      # aquí x es una variable global.
        x='changedinside'
    insideglobal()
    print("la x en ejemplo... es ",x)

def examplenonlocal():
    #global x          # Problema! conflicto con "non local".
    x='changedinexample'
    def insidenonlocal():
        nonlocal x    # aquí x una variable local de examplenonlocal
        x='changedinside'
    insidenonlocal()
    print("la x en ejemplo... es ",x)
```

In []:

```
x='notchanged'
examplenothing() # x no es cambiada en la primer función
print("global x es ",x) # x es cambiada de manera global
print(5*'-')
x='notchanged'
exampleglobal() # x no es cambiada en la primer función
print("global x es ",x) # x es cambiada de manera global
print(5*'-')
x='notchanged'
examplenonlocal()
print("global x es ",x) # x es cambiada de manera global
print(5*'-')
```

Añadir nuestra función al comando "help"

Si queremos añadir texto el cual será impreso cuando llamamos a la función `$\normalsize \color{green} \{ \textsf{ help } \} $`, simplemente debemos escribir una(s) línea(s) de texto luego de la definición de nuestra función.

In []:

```
def nicelydocumented():
    "Esta función devuelve TRUE siempre." # Comentario para la función help.
    "Esta línea no será vista" # Esta línea no se mostrara.
    return True
```


In []:

```
nicelydocumented()
```

In []:

```
help(nicelydocumented)
```

Como hemos visto en la definición de la función `Fibo(n)`, es posible escribir más de una línea de texto, utilizando cualquier método para escribir strings largos (i.e. triple comillas o la barra al final de cada línea).

Argumentos posicionales y argumentos asignados

En Python, las funciones, a priori, deben ser definidas con un número fijo de argumentos. Si nuestra función no recibe la cantidad correcta de argumentos obtendremos un error del tipo **TypeError**.

Podemos definir los argumentos de dos formas:

- (argumento) posicional: el orden de los argumentos es importante.
- (argumento) asignado: podemos asignar valores, ya sea por defecto, o mediante asignación directa en la llamada de la función.

In []:

```
print('c','g',end=" ** ",sep="**") # 'c' es un argumento posicional, mientras qu  
e,  
                                     # 'end'('endofthe[...]printline') y 'sep' son  
argumentos asignados.  
print('g','c')
```

También podemos dar valores por defecto a nuestros argumentos. Escribimos de la forma:

`def func(posarg_1, ..., posarg_r, kwarg_1 = vk_1, ..., kwarg_s = vk_s):`

los argumentos `posarg_1, ..., posarg_r` serán argumentos posicionales, sin valores por defecto, mientras que `vk_1, ..., vk_s` tendrán valores por defecto.

In [27]:

```
# Un simple ejemplo son solo argumentos posicionales  
def Displayingarguments1(a,b,c):  
    "Muestra los argumentos uno por linea"  
    print("argumento posicional a es ",a)  
    print("argumento posicional b es ",b)  
    print("argumento posicional c es ",c)
```

In []:

```
Displayingarguments1('si',(1,2,3),'no') # ok!
```

In []:

```
# No podemos llamar posicionales como argumentos asignados
Displayingarguments1((1,2,3), b='si',c='no')
```

In []:

```
# Debemos proveer algunos de los argumentos posicionales (primero)
Displayingarguments1('si',(1,2,3))
```

In []:

```
# Todos los que creamos necesarios
Displayingarguments1('si',(1,2,3),'no',3)
```

In []:

```
# No podemos asignar mas de un valor a b
Displayingarguments1('si',(1,2,3),'no',b=(1,2))
```

In []:

```
# No es posible poner argumentos posicionales luego de un argumento asignado
Displayingarguments1('si',b=(1,2,3),'no')
```

In []:

```
def Displayingarguments2(a,b,c,kw1='defautkw1',kw2='defautkw2',kw3='defautkw3'):
    # Un ejemplo con ambos.

    "Muestra los argumentos uno por linea"
    print("argumento posicional a es ",a)
    print("argumento posicional b es ",b)
    print("argumento posicional c es ",c)
    print("argumento asignado kw1 es ",kw1)
    print("argumento asignado kw2 es ",kw2)
    print("argumento asignado kw3 es ",kw3)
```

In []:

```
Displayingarguments2('si',(1,2,3),'no','algo',3,'nada') # Todos los argumentos s
on dados de manera posicional
```

In []:

```
Displayingarguments2('si',(1,2,3),'no') # Solo los argumentos posicionales son l
lamados,
                                         # los argumentos asignados obtendran val
ores por defecto
```

In []:

```
Displayingarguments2('si',(1,2,3),'no','algo', kw3='nada') # los argumentos pos
iccionales son llamados de esa manera
# kw1 es llamado com
o argumento posicional
# kw2 es dado su val
or por defecto
# kw3 es llamado com
o argumento asignado
```

In []:

```
# No es posible pasar argumentos por asignación, antes de los posicionales (o me
zclados)
Displayingarguments2('si',(1,2,3),'no', kw3='nada','algo')
```

In []:

```
def Displayingarguments3(a,b,c,kw=[]): # Ejemplo con un argumento mutable (l
a lista 'kw')
    "Muestra los argumentos uno por linea"
    print("argumento posicional a es ",a)
    print("argumento posicional b es ",b)
    print("argumento posicional c es ",c)
    print("argumento asignado kw es ",kw)
    kw.append(1)
```

In []:

```
Displayingarguments3(1,2,3) # El valor de 'kw' es cambiado!
```

In []:

```
def Displayingarguments4(a,b,c,kw=None):# Un ejemplo con argumentos mutables

    "Mostar los argumentos, uno por linea"
    if kw is None:
        kw=[]
    print("argumento posicional a es ",a)
    print("argumento posicional b es ",b)
    print("argumento posicional c es ",c)
    print("argumento asignado kw is ",kw)
    kw.append(1)
```

In []:

```
Displayingarguments4(1,2,3) # ok.
```

In []:

```
# Ejercicio : Escribir una función "cambio" que tiene un argumento posicional (un string) y otros dos argumentos
# la cual cambie las palabras las dos palabras recibidas como argumento en el string.
# Por defecto debe cambiar los espacios " ", por asteriscos "*".

def cambio(stringtobechanged,firstchar=' ',secchar='*'):
    res=''
    for c in stringtobechanged:
        if c==firstchar:
            res+=secchar
        else:
            res+=c
    return res
x='este es el string a ser modificado'
cambio(x,secchar='s')
```

Los nombres de los argumentos importantes merecen un nombre apropiado (descriptivo)

In []:

```
def reverseornot(stringofchar,rev=True):
    if rev:
        print(stringofchar[::-1])
    else:
        print(stringofchar)
reverseornot(x,rev=False)
```

In []:

```
lista=[x for x in range(0,20)]
type(range(0,20))
```

El comando Yield

Ya hemos visto el objeto `range`, el cual no contiene objetos en si mismo, pero tiene un '*tipo especial*'. Además de ser un objeto **iterable**, es decir, podremos recorrerlo usando un ciclo `for`, por ejemplo. Un rango, debe ser comprendido como una '*lista potencial*'.

La diferencia técnica radica en lo siguiente. Veamos la lista `L = [ 0, ..., 101000 - 1 ]`, es un objeto que ocupara mucha memoria, mientras que `range(101000 - 1)` será visto como: "*comenzar en 0 y contar hasta 10¹⁰⁰⁰-1*". Cuando recorremos la lista `L` usando un rango, *iteraremos sobre el rango*, es decir, se generaran las variables índices automáticamente y de a una, primero `x=0`, luego `x=1`, y así sucesivamente.

In []:

```
L=[i for i in range(10)]# Creación de la lista [0,1,2,3,4,5,6,7,8,9]
for x in L:                # Recorremos la lista: x=0, x=1,... x=9
    print(x,end=",")
print("")
for x in range(10): # x=0 luego x+=1, x+=1, ... hasta concluir en x=10
    print(x,end=",")
print("")
```

In []:

```
gen1=(x*x for x in range(10))
type(gen1)
gen1
```

El comando `yield` nos permite crear objetos más generales. Los llamaremos **generadores**.

In []:

```
gen1=(x*x for x in range(10))
def generateur(N):
    myrange=range(N)
    for i in myrange:
        print(10*"--")
        print("Paso: ",i)# Cuando generamos i*i?
        print(10*"--")
        yield i*i # El comando yield tiene el mismo 'status' que el comando return.
gen2=generateur(10)# No es una lista de cuadrados.
print("gen2 ha generado alguna salida?")
for i in gen2:
    print(i)
```